

# 1. Условный оператор if/else

## Что это?

Это конструкция, которая позволяет выполнять разные блоки кода в зависимости от условия.

```
#include <iostream>
using namespace std;

int main() {
    int age;
    cout << "Введите ваш возраст: ";
    cin >> age; // Пользователь вводит возраст

    // Блок if - проверяет условие
    if (age >= 18) { // ЕСЛИ возраст больше или равен 18
        cout << "Вы совершеннолетний" << endl;
    }
    // Блок else if - дополнительное условие
    else if (age >= 13) { // ИНАЧЕ ЕСЛИ возраст от 13 до 17
        cout << "Вы подросток" << endl;
    }
    // Блок else - выполняется, если все условия ложны
    else { // ИНАЧЕ (возраст меньше 13)
        cout << "Вы ребенок" << endl;
    }

    return 0;
}
```



## Разбор кода:

- `int age;` — объявляем переменную для хранения возраста
- `cin >> age;` — считываем число с клавиатуры
- `if (age >= 18)` — проверяем: возраст  $\geq 18$ ?

- `{ ... }` — блок кода, который выполняется если условие истинно
  - `else if (age >= 13)` — проверяется только если первое условие ложно
  - `else` — выполняется если ВСЕ предыдущие условия ложны
- 

## 2. Логические операции

### Операторы:

- `&&` — И (AND): оба условия должны быть истинны
- `||` — ИЛИ (OR): хотя бы одно условие должно быть истинно
- `!` — НЕ (NOT): инвертирует (меняет на противоположное)

```
#include <iostream>
using namespace std;

int main() {
    int age;
    bool hasTicket;

    cout << "Введите возраст: ";
    cin >> age;
    cout << "Есть ли билет? (1 - да, 0 - нет): ";
    cin >> hasTicket;

    // Пример с && (И)
    if (age >= 18 && hasTicket) { // И возраст ≥ 18, И есть билет
        cout << "Можно войти в клуб" << endl;
    } else {
        cout << "Нельзя войти в клуб" << endl;
    }

    // Пример с || (ИЛИ)
    if (age < 7 || age > 65) { // ИЛИ возраст < 7, ИЛИ возраст > 65
        cout << "Вы имеете право на льготы" << endl;
    }
```

```
// Пример с ! (НЕ)
if (!hasTicket) { // ЕСЛИ НЕ hasTicket (т.е. если билета
    cout << "Вам нужен билет!" << endl;
}

// Комбинированный пример
if ((age >= 12 && age <= 17) || (age >= 60 && hasTicket))
    // ИЛИ: (возраст от 12 до 17), ИЛИ: (возраст ≥ 60 И е
    cout << "Специальные условия" << endl;
}

return 0;
}
```

#### Таблица истинности:

| A     | B     | A && B | A    B | !A    |
|-------|-------|--------|--------|-------|
| true  | true  | true   | true   | false |
| true  | false | false  | true   | false |
| false | true  | false  | true   | true  |
| false | false | false  | false  | true  |

---

## 3. Оператор switch

### Что это?

Альтернатива цепочке if/else if, когда нужно проверить одну переменную на равенство с разными значениями.

```
#include <iostream>
using namespace std;

int main() {
    int dayNumber;
    cout << "Введите номер дня недели (1-7): ";
    cin >> dayNumber;

    switch(dayNumber) { // Проверяем значение переменной dayNumber
```

```
case 1:  // ЕСЛИ dayNumber == 1
        cout << "Понедельник" << endl;
        break;  // Выходим из switch

case 2:  // ЕСЛИ dayNumber == 2
        cout << "Вторник" << endl;
        break;

case 3:
        cout << "Среда" << endl;
        break;

case 4:
        cout << "Четверг" << endl;
        break;

case 5:
        cout << "Пятница" << endl;
        break;

case 6:
        cout << "Суббота" << endl;
        break;

case 7:
        cout << "Воскресенье" << endl;
        break;

default:  // Выполняется, если ни один case не подошел
        cout << "Неверный номер дня!" << endl;
        break;
}

return 0;
}
```

### Важные моменты:

- `break;` — обязателен (иначе выполнение "провалится" в следующий case)
- `default:` — необязателен, но хорошая практика

- switch работает только с целыми числами и символами
- 

## 4. Тернарный оператор ? :

### Что это?

Сокращенная форма if/else для простых присваиваний.

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cout << "Введите два числа: ";
    cin >> a >> b;

    // Стандартный if/else:
    int max;
    if (a > b) {
        max = a;
    } else {
        max = b;
    }

    // Тернарный оператор (то же самое в одной строке):
    int max2 = (a > b) ? a : b;
    // Читается так: ЕСЛИ (a > b) ТО a ИНАЧЕ b

    cout << "Максимум (if/else): " << max << endl;
    cout << "Максимум (тернарный): " << max2 << endl;

    // Еще пример:
    string result = (a == b) ? "числа равны" : "числа разные"
    cout << result << endl;

    return 0;
}
```



**Синтаксис:** условие ? значение\_если\_истина : значение\_если\_ложь

---



## ПРИМЕР 1: Простой калькулятор

```
#include <iostream>
using namespace std;

int main() {
    double num1, num2, result;
    char operation;

    // Ввод данных
    cout << "Введите первое число: ";
    cin >> num1; // Считываем первое число

    cout << "Введите операцию (+, -, *, /): ";
    cin >> operation; // Считываем символ операции

    cout << "Введите второе число: ";
    cin >> num2; // Считываем второе число

    // Выполнение операции с проверкой
    if (operation == '+') {
        result = num1 + num2; // Сложение
        cout << num1 << " + " << num2 << " = " << result << endl;
    }
    else if (operation == '-') {
        result = num1 - num2; // Вычитание
        cout << num1 << " - " << num2 << " = " << result << endl;
    }
    else if (operation == '*') {
        result = num1 * num2; // Умножение
        cout << num1 << " * " << num2 << " = " << result << endl;
    }
    else if (operation == '/') {
        // Проверка деления на ноль!
        if (num2 != 0) { // Если делитель не равен нулю
            result = num1 / num2; // Деление
            cout << num1 << " / " << num2 << " = " << result << endl;
        } else {
```

```
        cout << "Ошибка: деление на ноль!" << endl;
    }
}
else { // Если введена неподдерживаемая операция
    cout << "Ошибка: неизвестная операция!" << endl;
}

return 0;
}
```

---

## ПРИМЕР 2: Проверка введенных данных

```
#include <iostream>
using namespace std;

int main() {
    int score;

    cout << "Введите оценку (0-100): ";
    cin >> score;

    // Проверка корректности ввода
    if (score < 0 || score > 100) { // Если оценка < 0 ИЛИ >
        cout << "Ошибка: оценка должна быть от 0 до 100!" << endl;
        return 1; // Завершаем программу с кодом ошибки
    }

    // Определение оценки по буквенной системе
    char grade;

    if (score >= 90) {
        grade = 'A';
    }
    else if (score >= 80) { // Выполнится только если score >= 80
        grade = 'B';
    }
    else if (score >= 70) { // Выполнится только если score >= 70
        grade = 'C';
    }
    else if (score >= 60) { // Выполнится только если score >= 60
        grade = 'D';
    }
    else {
        grade = 'F';
    }

    cout << "Ваша оценка: " << grade << endl;
}
```

```
        grade = 'D';
    }
    else { // Выполнится только если score < 60
        grade = 'F';
    }

    cout << "Ваша оценка: " << grade << endl;

    // Дополнительная проверка с тернарным оператором
    string message = (grade == 'F') ? "Нужно подтянуть знания" : "";
    cout << message << endl;

    return 0;
}
```

---



### ПРИМЕР 3: Определение времени суток

```
#include <iostream>
using namespace std;

int main() {
    int hour;

    cout << "Введите текущий час (0-23): ";
    cin >> hour;

    // Проверка корректности
    if (hour < 0 || hour > 23) {
        cout << "Неверное время! Часы должны быть от 0 до 23." << endl;
        return 1;
    }

    // Определение времени суток
    cout << "Сейчас ";

    if (hour >= 5 && hour < 12) { // С 5:00 до 11:59
        cout << "утро";
    }
    else if (hour >= 12 && hour < 17) { // С 12:00 до 16:59
        cout << "день";
    }
}
```



```
}  
else if (hour >= 17 && hour < 22) { // с 17:00 до 21:59  
    cout << "вечер";  
}  
else { // с 22:00 до 4:59  
    cout << "ночь";  
}  
  
cout << "." << endl;  
  
// Использование switch для конкретных часов  
switch(hour) {  
    case 0:  
        cout << "Полночь!" << endl;  
        break;  
    case 12:  
        cout << "Полдень!" << endl;  
        break;  
    case 8:  
    case 9:  
    case 10: // Для 8, 9 и 10 выполнится одно и то же  
        cout << "Идеальное время для работы!" << endl;  
        break;  
}  
  
return 0;  
}
```

---



## Ключевые моменты для запоминания:

### 1. Синтаксис if:

```
if (условие) {  
    // код выполнится, если условие ИСТИННО  
}
```

### 2. Цепочка условий:

```
if (условие1) {  
    // код 1  
} else if (условие2) {  
    // код 2 (только если условие1 ложно, а условие2 истинно)  
} else {  
    // код 3 (если все условия ложны)  
}
```

### 3. Логические операторы:

- `&&` — как "и" в русском: "если идет дождь **И** у меня есть зонт"
- `||` — как "или" в русском: "если у меня есть деньги **ИЛИ** карта"
- `!` — отрицание: "если **НЕ** идет дождь"

### 4. switch vs if:

- Используйте `switch` когда проверяете одну переменную на конкретные значения
- Используйте `if` для сложных условий и диапазонов

### 5. Всегда проверяйте ввод пользователя!

```
if (cin.fail()) { // Проверка на некорректный ввод  
    cout << "Ошибка ввода!" << endl;  
}
```



### Советы для начинающих:

1. Всегда используйте фигурные скобки `{ }`, даже если код одна строка
2. Пишите понятные условия: `if (age >= 18)` лучше чем `if (age > 17)`

3. **Проверяйте граничные значения:** что будет при 0, при отрицательных числах?
  4. **Комментируйте сложные условия**
  5. **Тестируйте все возможные варианты**
- 

## Практическое задание:

Создайте программу, которая:

1. Запрашивает у пользователя температуру
2. Определяет: холодно ( $< 10^{\circ}\text{C}$ ), тепло ( $10\text{-}25^{\circ}\text{C}$ ), жарко ( $> 25^{\circ}\text{C}$ )
3. Проверяет, корректно ли введены данные (температура не ниже  $-100$  и не выше  $+100$ )
4. Дополнительно: если температура ниже  $0^{\circ}$ , выводит "Осторожно, лед!"

# Глубокое погружение в условные операторы и логические операции в C++ - Полный сборник листингов программ по условным операторам в C++

## Программа 1: Базовые примеры if/else

### 1.1. Минимальный пример if/else

```
// =====  
// Файл: 01_basic_if_else.cpp  
// Описание: Простейшие формы условных операторов
```

```
// =====

#include <iostream> // Подключаем библиотеку для ввода/вывода
using namespace std; // Используем стандартное пространство имен

int main() {          // Главная функция - точка входа в программу
    // Пример 1: Простой if без else
    int x = 10;        // Объявляем целочисленную переменную x и присваиваем ей значение 10

    // Базовая форма if
    if (x > 0) {        // Условие: если x больше 0
        // Блок кода, который выполняется если условие истинно
        cout << "x положительное число" << endl; // Выводим сообщение
    }
    // endl - манипулятор конца строки (переводит курсор на новую строку)

    // Пример 2: if с else
    int y = -5;         // Объявляем переменную y с отрицательным значением

    if (y > 0) {         // Условие: если y больше 0
        cout << "y положительное" << endl;
    } else {             // else - выполняется если условие ложно
        cout << "y не положительное" << endl;
    }

    // Пример 3: if-else if-else цепочка
    int score = 85;     // Баллы студента

    if (score >= 90) {   // Первое условие
        cout << "Оценка A" << endl;
    }
    else if (score >= 80) { // Проверяется только если первое условие ложно
        cout << "Оценка B" << endl;
    }
    else if (score >= 70) { // Проверяется только если все предыдущие условия ложны
        cout << "Оценка C" << endl;
    }
    else {               // Выполняется если все условия ложны
        cout << "Оценка F" << endl;
    }

    // Пример 4: if без фигурных скобок (только для одной инструкции)
    int a = 5, b = 10;
```

```

    if (a < b)                // Если условие истинно
        cout << "a меньше b" << endl; // Выполнится только э

// ОПАСНО: без скобок можно допустить ошибку
if (a > b)
    cout << "Это выведется если a > b" << endl;
    cout << "А это выведется ВСЕГДА!" << endl; // Вне if

return 0; // Возвращаем 0 - код успешного завершения прог
}

```

## 1.2. Вложенные условия и висячий else

```

// =====
// Файл: 02_nested_if_dangling_else.cpp
// Описание: Проблема висячего else и вложенные условия
// =====

#include <iostream>
using namespace std;

int main() {
    // Проблема "висячего else" (dangling else problem)
    int x = 10, y = 5;

    // Пример 1: Неоднозначность
    cout << "Пример 1 (неоднозначный код):" << endl;
    if (x > 0)
        if (y > 0)
            cout << "Обе переменные положительные" << endl;
    else // К какому if относится этот else?
        cout << "Это сообщение выведется неожиданно!" << endl;
    // По правилам C++ else относится к ближайшему if

    // Пример 2: Правильное оформление со скобками
    cout << "\nПример 2 (правильное оформление):" << endl;
    if (x > 0) {
        if (y > 0) {
            cout << "Обе переменные положительные" << endl;
        }
    }
}

```

```

    } else {
        cout << "x не положительный" << endl;
    }

    // Пример 3: Глубокая вложенность (антипаттерн!)
    cout << "\nПример 3 (глубокая вложенность):" << endl;
    int a = 1, b = 2, c = 3, d = 4;

    if (a > 0) {
        if (b > 0) {
            if (c > 0) {
                if (d > 0) {
                    cout << "Все переменные положительные" <<
                } else {
                    cout << "d не положительная" << endl;
                }
            } else {
                cout << "c не положительная" << endl;
            }
        } else {
            cout << "b не положительная" << endl;
        }
    } else {
        cout << "a не положительная" << endl;
    }

    return 0;
}

```

## Программа 2: Логические операции

### 2.1. Базовые логические операции

```

// =====
// Файл: 03_logical_operations.cpp
// Описание: Логические операторы &&, ||, !
// =====

#include <iostream>
using namespace std;

```

```
// Вспомогательная функция для демонстрации short-circuit
bool check(bool value, const string& name) {
    cout << "Проверка " << name << ": " << (value ? "true" : "false") << endl;
    return value;
}

int main() {
    cout << "=== ДЕМОНСТРАЦИЯ ЛОГИЧЕСКИХ ОПЕРАЦИЙ ===\n" << endl;

    // Пример 1: Оператор И (&&)
    cout << "1. Оператор И (&&):" << endl;
    bool hasTicket = true;
    bool hasMoney = true;
    bool hasTime = false;

    // ОБА условия должны быть истинны
    if (hasTicket && hasMoney) {
        cout << "Можно пойти в кино" << endl;
    } else {
        cout << "Нельзя пойти в кино" << endl;
    }

    // Пример 2: Оператор ИЛИ (||)
    cout << "\n2. Оператор ИЛИ (||):" << endl;
    bool isWeekend = true;
    bool isHoliday = false;

    // ХОТЯ БЫ ОДНО условие должно быть истинно
    if (isWeekend || isHoliday) {
        cout << "Можно отдохнуть!" << endl;
    }

    // Пример 3: Оператор НЕ (!)
    cout << "\n3. Оператор НЕ (!):" << endl;
    bool doorLocked = true;

    if (!doorLocked) { // Если НЕ doorLocked
        cout << "Дверь открыта" << endl;
    } else {
        cout << "Дверь закрыта" << endl;
    }
}
```

```
// Пример 4: Комбинирование операторов
cout << "\n4. Комбинирование операторов:" << endl;
int age = 25;
bool hasLicense = true;
bool isSober = true;

// Можно водить если: возраст >= 18 И есть права И трезв
if (age >= 18 && hasLicense && isSober) {
    cout << "Можно водить машину" << endl;
}

// Пример 5: Short-circuit evaluation (ленивое вычисление
cout << "\n5. Short-circuit evaluation:" << endl;

// Для &&: если левая часть ложна, правая НЕ вычисляется
cout << "Проверка с && (первое false):" << endl;
if (check(false, "condition1") && check(true, "condition2")) {
    // Второй check не выполнится!
}

cout << "\nПроверка с && (первое true):" << endl;
if (check(true, "condition1") && check(false, "condition2")) {
    // Оба check выполняются
}

cout << "\nПроверка с || (первое true):" << endl;
if (check(true, "condition1") || check(true, "condition2")) {
    // Второй check не выполнится!
}

// Пример 6: Преобразование типов в условиях
cout << "\n6. Преобразование типов в условиях:" << endl;
int value = 5;           // ненулевое значение → true
int zero = 0;           // ноль → false
int* ptr = nullptr;     // нулевой указатель → false

if (value) {
    cout << "value считается истиной" << endl;
}

if (!zero) {
    cout << "zero считается ложью" << endl;
}
```



```

    if (!ptr) {
        cout << "nullptr считается ложью" << endl;
    }

    return 0;
}

```

## 2.2. Сложные логические выражения

```

// =====
// Файл: 04_complex_logical_expressions.cpp
// Описание: Сложные условия и приоритет операций
// =====

#include <iostream>
using namespace std;

int main() {
    cout << "=== СЛОЖНЫЕ ЛОГИЧЕСКИЕ ВЫРАЖЕНИЯ ===\n" << endl;

    // Пример 1: Приоритет операций
    cout << "1. Приоритет логических операций:" << endl;
    cout << "Высший: ! (НЕ)" << endl;
    cout << "Средний: && (И)" << endl;
    cout << "Низший: || (ИЛИ)" << endl;

    bool a = true, b = false, c = true;

    // Без скобок: && имеет приоритет над ||
    // true && false || true = false || true = true
    bool result1 = a && b || c;
    cout << "a && b || c = " << (result1 ? "true" : "false") << endl;

    // Со скобками: меняем порядок
    // true && (false || true) = true && true = true
    bool result2 = a && (b || c);
    cout << "a && (b || c) = " << (result2 ? "true" : "false") << endl;

    // Пример 2: Правила де Моргана
    cout << "\n2. Правила де Моргана:" << endl;
}

```

```

int x = 5, y = 15;

// Оригинальное условие
if (!(x > 0 && y < 10)) {
    cout << "!(x > 0 && y < 10) истинно" << endl;
}

// Эквивалент по де Моргану: !(A && B) = !A || !B
if (!(x > 0) || !(y < 10)) { // x <= 0 || y >= 10
    cout << "!A || !B также истинно" << endl;
}

// Пример 3: Сложное условие с диапазонами
cout << "\n3. Проверка на вхождение в диапазон:" << endl;
int score = 75;

// Проверка что score в диапазоне [0, 100]
if (score >= 0 && score <= 100) {
    cout << "Балл " << score << " находится в допустимом ,
}

// Проверка что score НЕ в диапазоне [60, 80]
if (!(score >= 60 && score <= 80)) {
    cout << "Балл " << score << " НЕ в диапазоне [60, 80]"
}

// Пример 4: XOR через базовые операторы
cout << "\n4. Реализация XOR (исключающего ИЛИ):" << endl;
bool p = true, q = false;

// XOR: истинно когда значения РАЗНЫЕ
// (p || q) && !(p && q)
bool xor_result = (p || q) && !(p && q);
cout << "p = " << p << ", q = " << q << ", XOR = " << xor_

// Пример 5: Условные выражения с побочными эффектами
cout << "\n5. Побочные эффекты в условиях (ОПАСНО!):" << endl;
int counter = 0;

// ОПАСНО: порядок вычисления не гарантирован!
// if (counter++ && counter++) { // Непредсказуемый резул

// Безопасный вариант:

```

```
    int temp1 = counter++;
    int temp2 = counter++;
    if (temp1 && temp2) {
        cout << "Безопасный вариант работает" << endl;
    }

    return 0;
}
```

## Программа 3: Тернарный оператор

### 3.1. Базовое использование тернарного оператора

```
// =====
// Файл: 05_ternary_operator.cpp
// Описание: Тернарный оператор ?:
// =====

#include <iostream>
#include <string>
using namespace std;

int main() {
    cout << "=== ТЕРНАРНЫЙ ОПЕРАТОР ? : ===\n" << endl;

    // Пример 1: Простейшая форма
    cout << "1. Простейшая форма:" << endl;
    int a = 10, b = 20;

    // Традиционный if-else
    int max_if;
    if (a > b) {
        max_if = a;
    } else {
        max_if = b;
    }

    // Тернарный оператор
    int max_ternary = (a > b) ? a : b;
```

```
cout << "if-else: max = " << max_if << endl;
cout << "тернарный: max = " << max_ternary << endl;

// Пример 2: Использование в выражениях
cout << "\n2. Использование в выражениях:" << endl;
int x = 5;

// Можно использовать прямо в выводе
cout << "x = " << x << ", это число "
    << ((x % 2 == 0) ? "четное" : "нечетное") << endl;

// И в вычислениях
int y = (x > 0) ? x * 2 : -x * 2;
cout << "y = " << y << endl;

// Пример 3: Определение типа результата
cout << "\n3. Определение типа результата:" << endl;
int int_val = 10;
double double_val = 3.14;

// Компилятор определяет общий тип
auto result = (x > 0) ? double_val : int_val;
cout << "Тип результата: " << typeid(result).name()
    << ", значение: " << result << endl;

// Пример 4: Вложенные тернарные операторы
cout << "\n4. Вложенные тернарные операторы (осторожно!):" << endl;
int score = 85;

// Эмуляция if-else if-else цепочки
string grade = (score >= 90) ? "A" :
    (score >= 80) ? "B" :
    (score >= 70) ? "C" :
    (score >= 60) ? "D" : "F";

cout << "Балл: " << score << ", оценка: " << grade << endl;

// Пример 5: Sequence points и тернарный оператор
cout << "\n5. Sequence points:" << endl;
int m = 5, n = 10;

// Тернарный оператор гарантирует порядок вычисления
int r = (m++ > 3) ? n++ : --n;
```

```

    cout << "m = " << m << ", n = " << n << ", r = " << r << "\n";

    // Эквивалентный код с if-else:
    int temp;
    if (m++ > 3) { // Здесь m уже 6!
        temp = n++;
    } else {
        temp = --n;
    }
    int r2 = temp;

    return 0;
}

```

### 3.2. Constexpr и тернарный оператор

```

// =====
// Файл: 06_ternary_constexpr.cpp
// Описание: Тернарный оператор в compile-time
// =====

#include <iostream>
#include <type_traits>
using namespace std;

// Пример 1: Compile-time вычисления
template<typename T>
constexpr const char* type_name() {
    // Тернарный оператор может использоваться в constexpr функции
    return is_integral<T>::value ? "integral" :
           is_floating_point<T>::value ? "floating point" :
           is_pointer<T>::value ? "pointer" :
           "other";
}

int main() {
    cout << "=== ТЕРНАРНЫЙ ОПЕРАТОР В COMPILE-TIME ===\n" << "\n";

    // Пример 1: Constexpr переменные
    constexpr bool is_64bit = (sizeof(void*) == 8);
    constexpr int buffer_size = is_64bit ? 1024 : 512;
}

```

```

cout << "Система " << (is_64bit ? "64-битная" : "32-битная") << endl;
cout << "Размер буфера: " << buffer_size << " байт" << endl;

// Пример 2: constexpr функция с тернарным оператором
constexpr int max_constexpr(int a, int b) {
    return (a > b) ? a : b; // Тернарный оператор в constexpr
}

constexpr int max_val = max_constexpr(10, 20);
cout << "Max (constexpr): " << max_val << endl;

// Пример 3: Определение типов во время компиляции
cout << "\nТипы во время компиляции:" << endl;
cout << "int: " << type_name<int>() << endl;
cout << "double: " << type_name<double>() << endl;
cout << "int*: " << type_name<int*>() << endl;
cout << "string: " << type_name<string>() << endl;

// Пример 4: Выбор реализации на основе типа
auto process_value = [](auto value) {
    // C++17: if constexpr с тернарным оператором
    if constexpr (is_integral<decltype(value)>::value) {
        return value * 2;
    } else if constexpr (is_floating_point<decltype(value)>::value) {
        return value / 2.0;
    } else {
        return value; // для других типов
    }
};

cout << "\nОбработка значений:" << endl;
cout << "process_value(10) = " << process_value(10) << endl;
cout << "process_value(3.14) = " << process_value(3.14) << endl;

return 0;
}

```

## Программа 4: Оператор switch

## 4.1. Базовое использование switch

```
// =====  
// Файл: 07_switch_basic.cpp  
// Описание: Базовое использование оператора switch  
// =====  
  
#include <iostream>  
#include <string>  
using namespace std;  
  
// Перечисление для дней недели  
enum class Day {  
    Monday,  
    Tuesday,  
    Wednesday,  
    Thursday,  
    Friday,  
    Saturday,  
    Sunday  
};  
  
int main() {  
    cout << "=== ОПЕРАТОР SWITCH ===\n" << endl;  
  
    // Пример 1: Switch с целыми числами  
    cout << "1. Switch с целыми числами:" << endl;  
    int day_number = 3;  
  
    switch (day_number) { // Выражение для проверки  
        case 1: // если day_number == 1  
            cout << "Понедельник" << endl;  
            break; // Выход из switch  
  
        case 2:  
            cout << "Вторник" << endl;  
            break;  
  
        case 3:  
            cout << "Среда" << endl;  
            break;
```

```
case 4:
    cout << "Четверг" << endl;
    break;

case 5:
    cout << "Пятница" << endl;
    break;

case 6:
case 7: // Объединение нескольких case
    cout << "Выходной" << endl;
    break;

default: // Выполняется если ни один case не совпал
    cout << "Неверный номер дня" << endl;
}
```

```
// Пример 2: Switch с символами
cout << "\n2. Switch с символами:" << endl;
char grade = 'B';
```

```
switch (grade) {
    case 'A':
    case 'a':
        cout << "Отлично!" << endl;
        break;

    case 'B':
    case 'b':
        cout << "Хорошо" << endl;
        break;

    case 'C':
    case 'c':
        cout << "Удовлетворительно" << endl;
        break;

    default:
        cout << "Неудовлетворительно" << endl;
}
```

```
// Пример 3: Switch с enum
cout << "\n3. Switch с перечислениями (enum):" << endl;
```



```
Day today = Day::Wednesday;
```

```
switch (today) {
    case Day::Monday:
        cout << "Начало недели" << endl;
        break;

    case Day::Friday:
        cout << "Сколько выходные!" << endl;
        break;

    case Day::Saturday:
    case Day::Sunday:
        cout << "Выходной день" << endl;
        break;

    default:
        cout << "Будний день" << endl;
}
```

```
// Пример 4: Отсутствие break (fall-through)
cout << "\n4. Fall-through (отсутствие break):" << endl;
int option = 2;
```

```
switch (option) {
    case 1:
        cout << "Выполняется case 1" << endl;
        // НЕТ break - выполнение "проваливается" в case :
    case 2:
        cout << "Выполняется case 2" << endl;
        // НЕТ break - проваливается в case 3!
    case 3:
        cout << "Выполняется case 3" << endl;
        break; // Выход из switch
    default:
        cout << "Default" << endl;
}
```

```
// Пример 5: Использование в функциях
cout << "\n5. Switch в функциях:" << endl;
auto get_day_name = [](int day) -> string {
    switch (day) {
        case 1: return "Monday";
```

```

        case 2: return "Tuesday";
        case 3: return "Wednesday";
        case 4: return "Thursday";
        case 5: return "Friday";
        case 6: return "Saturday";
        case 7: return "Sunday";
        default: return "Invalid day";
    }
};

cout << "День 3: " << get_day_name(3) << endl;
cout << "День 8: " << get_day_name(8) << endl;

return 0;
}

```

## 4.2. Оптимизация switch и ограничения

```

// =====
// Файл: 08_switch_optimization.cpp
// Описание: Ограничения и оптимизации switch
// =====

#include <iostream>
#include <random>
using namespace std;

int main() {
    cout << "=== ОПТИМИЗАЦИЯ И ОГРАНИЧЕНИЯ SWITCH ===\n" << endl;

    // Пример 1: Константные выражения в case
    cout << "1. Константные выражения в case:" << endl;

    constexpr int MIN_VALUE = 1;
    constexpr int MAX_VALUE = 10;

    int value = 5;

    switch (value) {
        case MIN_VALUE: // ОК: MIN_VALUE - константное выражение
            cout << "Минимальное значение" << endl;

```

```
        break;

    case MAX_VALUE: // OK: MAX_VALUE - константное выражение
        cout << "Максимальное значение" << endl;
        break;

    case 2 + 3: // OK: выражение, вычисляемое на этапе компиляции
        cout << "Значение 5" << endl;
        break;

    // ОШИБКА КОМПИЛЯЦИИ (раскомментировать для проверки)
    // case rand(): // НЕ константное выражение!
    //     cout << "Случайное значение" << endl;
    //     break;

    default:
        cout << "Другое значение" << endl;
}

// Пример 2: Оптимизация компилятором (jump tables)
cout << "\n2. Оптимизация плотных диапазонов:" << endl;

// ХОРОШО для оптимизации: плотный диапазон значений
int dense_value = 2;

switch (dense_value) {
    case 0: cout << "Ноль"; break;
    case 1: cout << "Один"; break;
    case 2: cout << "Два"; break;
    case 3: cout << "Три"; break;
    case 4: cout << "Четыре"; break;
    default: cout << "Другое";
}
cout << endl;

// Компилятор может создать jump table:
// addresses = {case0_addr, case1_addr, case2_addr, ...}
// goto addresses[value] // если value в диапазоне 0-4

// Пример 3: Разрозненные значения (менее эффективно)
cout << "\n3. Разрозненные значения:" << endl;
int sparse_value = 100;
```

```
switch (sparse_value) {
    case 1: cout << "Один"; break;
    case 100: cout << "Сто"; break;
    case 1000: cout << "Тысяча"; break;
    default: cout << "Другое";
}
cout << endl;

// Для разрозненных значений компилятор генерирует
// цепочку if-else сравнений вместо jump table

// Пример 4: Сравнение с if-else цепочкой
cout << "\n4. Сравнение switch и if-else:" << endl;
int test_value = 2;

// Switch версия
cout << "Switch: ";
switch (test_value) {
    case 0: cout << "Zero"; break;
    case 1: cout << "One"; break;
    case 2: cout << "Two"; break;
    default: cout << "Other";
}
cout << endl;

// Эквивалентная if-else цепочка
cout << "if-else: ";
if (test_value == 0) {
    cout << "Zero";
} else if (test_value == 1) {
    cout << "One";
} else if (test_value == 2) {
    cout << "Two";
} else {
    cout << "Other";
}
cout << endl;

// Пример 5: Вложенные switch
cout << "\n5. Вложенные switch:" << endl;
int outer = 1, inner = 2;

switch (outer) {
```

```

        case 1:
            cout << "Внешний case 1" << endl;
            switch (inner) { // Вложенный switch
                case 1: cout << "    Внутренний case 1"; break;
                case 2: cout << "    Внутренний case 2"; break;
            }
            break;

        case 2:
            cout << "Внешний case 2" << endl;
            break;
    }

    return 0;
}

```

## Программа 5: Паттерны и антипаттерны

### 5.1. Guard Clause и оптимизация условий

```

// =====
// Файл: 09_patterns_antipatterns.cpp
// Описание: Паттерны и антипаттерны условий
// =====

#include <iostream>
#include <memory>
#include <vector>
using namespace std;

// Класс для демонстрации
class Data {
private:
    vector<int> values;
    bool valid;

public:
    Data() : valid(false) {}

    Data(const vector<int>& v) : values(v), valid(true) {}

```

```

    bool is_valid() const { return valid; }
    bool has_data() const { return !values.empty(); }
    size_t size() const { return values.size(); }
    void process() const {
        cout << "Обработка " << values.size() << " элементов"
    }
};

```

```

// АНТИПАТТЕРН: Глубокая вложенность (Pyramid of Doom)
void process_data_bad(Data* data) {
    cout << "\nАНТИПАТТЕРН: Глубокая вложенность" << endl;

    if (data != nullptr) { // Уровень 1
        if (data->is_valid()) { // Уровень 2
            if (data->has_data()) { // Уровень 3
                if (data->size() > 0) { // Уровень 4
                    data->process(); // Основная логика
                } else {
                    cout << "Нет данных для обработки" << endl;
                }
            } else {
                cout << "Данные пустые" << endl;
            }
        } else {
            cout << "Данные невалидны" << endl;
        }
    } else {
        cout << "Указатель nullptr" << endl;
    }
}

```

```

// ПАТТЕРН: Guard Clause (ранний возврат)
void process_data_good(Data* data) {
    cout << "\nПАТТЕРН: Guard Clause" << endl;

    // Проверки в начале функции, возврат при ошибках
    if (data == nullptr) {
        cout << "Указатель nullptr" << endl;
        return; // Ранний возврат
    }

    if (!data->is_valid()) {

```

```
        cout << "Данные невалидны" << endl;
        return;
    }

    if (!data->has_data()) {
        cout << "Данные пустые" << endl;
        return;
    }

    if (data->size() == 0) {
        cout << "Нет данных для обработки" << endl;
        return;
    }

    // Основная логика - без вложенности!
    data->process();
}

// ПАТТЕРН: Флэттенинг условий
void check_access(int age, bool has_id, bool has_ticket) {
    cout << "\nПАТТЕРН: Флэттенинг условий" << endl;

    // Вместо вложенных if
    if (age < 18) {
        cout << "Несовершеннолетний - доступ запрещен" << endl;
        return;
    }

    if (!has_id) {
        cout << "Нет удостоверения - доступ запрещен" << endl;
        return;
    }

    if (!has_ticket) {
        cout << "Нет билета - доступ запрещен" << endl;
        return;
    }

    cout << "Доступ разрешен" << endl;
}

// ПАТТЕРН: Замена switch на полиморфизм
class Animal {
```

```
public:
    virtual ~Animal() = default;
    virtual void make_sound() const = 0;
};

class Dog : public Animal {
public:
    void make_sound() const override {
        cout << "Woof!" << endl;
    }
};

class Cat : public Animal {
public:
    void make_sound() const override {
        cout << "Meow!" << endl;
    }
};

class Bird : public Animal {
public:
    void make_sound() const override {
        cout << "Tweet!" << endl;
    }
};

// АНТИПАТТЕРН: Большой switch
void make_sound_switch(int animal_type) {
    cout << "\nАНТИПАТТЕРН: Большой switch" << endl;

    switch (animal_type) {
        case 0: cout << "Woof!" << endl; break;
        case 1: cout << "Meow!" << endl; break;
        case 2: cout << "Tweet!" << endl; break;
        // ... и так для 100 видов животных
        default: cout << "Unknown animal" << endl;
    }
}

// ПАТТЕРН: Полиморфизм вместо switch
void make_sound_polymorphic(const Animal& animal) {
    cout << "\nПАТТЕРН: Полиморфизм" << endl;
    animal.make_sound();
}
```



```

}

int main() {
    cout << "=== ПАТТЕРНЫ И АНТИПАТТЕРНЫ УСЛОВИЙ ===\n" << endl;

    // Создаем тестовые данные
    vector<int> test_data = {1, 2, 3, 4, 5};
    Data valid_data(test_data);
    Data invalid_data;

    // Тестируем антипаттерн и паттерн
    process_data_bad(&valid_data);
    process_data_bad(nullptr);

    process_data_good(&valid_data);
    process_data_good(nullptr);
    process_data_good(&invalid_data);

    // Тестируем флэттенинг
    check_access(20, true, true);    // Все условия выполнены
    check_access(16, true, true);    // Возраст не подходит
    check_access(20, false, true);   // Нет удостоверения

    // Тестируем полиморфизм vs switch
    make_sound_switch(0);    // Собака через switch

    Dog dog;
    Cat cat;
    Bird bird;

    make_sound_polymorphic(dog);
    make_sound_polymorphic(cat);
    make_sound_polymorphic(bird);

    return 0;
}

```

## 5.2. Branch prediction и оптимизации

```

// =====
// Файл: 10_branch_prediction.cpp

```

```
// Описание: Оптимизация ветвлений и предсказание
// =====

#include <iostream>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>
using namespace std;
using namespace chrono;

// Функция для тестирования производительности
template<typename Func>
double measure_time(Func f, int iterations = 1000000) {
    auto start = high_resolution_clock::now();
    for (int i = 0; i < iterations; ++i) {
        f();
    }
    auto end = high_resolution_clock::now();
    return duration_cast<duration<double>>(end - start).count
}

int main() {
    cout << "=== BRANCH PREDICTION И ОПТИМИЗАЦИИ ===\n" << endl;

    const int SIZE = 10000;
    vector<int> data(SIZE);

    // Заполняем случайными числами
    random_device rd;
    mt19937 gen(rd());
    uniform_int_distribution<> dis(0, 100);

    for (int& val : data) {
        val = dis(gen);
    }

    // Пример 1: Предсказуемые vs непредсказуемые ветвления
    cout << "1. Предсказуемые vs непредсказуемые ветвления:" << endl;

    // Сортируем данные для предсказуемых ветвлений
    vector<int> sorted_data = data;
    sort(sorted_data.begin(), sorted_data.end());
```

```

int count1 = 0, count2 = 0;

// Непредсказуемые ветвления (случайный порядок)
auto unpredictable = [&]() {
    int sum = 0;
    for (int val : data) {
        if (val < 50) { // 50/50 вероятность - плохо для
            sum += val;
        }
    }
    return sum;
};

// Предсказуемые ветвления (отсортированный массив)
auto predictable = [&]() {
    int sum = 0;
    for (int val : sorted_data) {
        if (val < 50) { // Сначала все true, потом все false
            sum += val; // Легко предсказать
        }
    }
    return sum;
};

double time_unpredictable = measure_time(unpredictable, 1000);
double time_predictable = measure_time(predictable, 1000);

cout << "Непредсказуемые ветвления: " << time_unpredictable << endl;
cout << "Предсказуемые ветвления: " << time_predictable << endl;
cout << "Разница: " << (time_unpredictable / time_predictable) << endl;

// Пример 2: Оптимизация логических выражений
cout << "\n2. Оптимизация логических выражений:" << endl;

// Демонстрация short-circuit оптимизации
auto expensive_check = []() -> bool {
    // Имитация дорогой операции
    volatile int x = 0;
    for (int i = 0; i < 1000; ++i) {
        x += i;
    }
    return false;
};

```

```

};

auto cheap_check = []() -> bool {
    return false; // Быстрая проверка
};

// Медленный вариант: дорогая операция выполняется всегда
cout << "Порядок условий имеет значение!" << endl;

// Пример 3: Замена условий битовыми операциями
cout << "\n3. Замена условий битовыми операциями:" << endl;

// Традиционный min
auto min_traditional = [](int a, int b) -> int {
    if (a < b) return a;
    else return b;
};

// Min без ветвлений (битовые операции)
auto min_bitwise = [](int a, int b) -> int {
    // mask = -1 если a < b, иначе 0
    int mask = (a - b) >> (sizeof(int) * 8 - 1);
    return (a & ~mask) | (b & mask);
};

// Тестируем
int a = 10, b = 20;
cout << "min_traditional(" << a << ", " << b << ") = "
    << min_traditional(a, b) << endl;
cout << "min_bitwise(" << a << ", " << b << ") = "
    << min_bitwise(a, b) << endl;

// Пример 4: Подсказки компилятору (GCC/Clang)
cout << "\n4. Подсказки компилятору:" << endl;

// Встроенные функции для подсказок branch prediction
bool condition = true; // Предполагаем что условие чаще

// GCC/Clang: __builtin_expect
// if (__builtin_expect(condition, 1)) {
//     // Компилятор разместит этот код как более вероятный
// }

```

```

// C++20: [[likely]] и [[unlikely]]
if (condition) [[likely]] {
    cout << "Вероятная ветка (likely)" << endl;
} else [[unlikely]] {
    cout << "Маловероятная ветка (unlikely)" << endl;
}

// Пример 5: Избегание ветвлений в критических циклах
cout << "\n5. Избегание ветвлений:" << endl;

// Плохо: ветвление внутри цикла
auto sum_with_branch = [](const vector<int>& vec) {
    int sum = 0;
    for (int val : vec) {
        if (val > 0) {
            sum += val;
        }
    }
    return sum;
};

// Лучше: избегаем ветвления (пример)
auto sum_without_branch = [](const vector<int>& vec) {
    int sum = 0;
    for (int val : vec) {
        // Используем битовые операции вместо if
        // mask = 0 если val <= 0, иначе -1
        int mask = ~(val >> (sizeof(int) * 8 - 1));
        sum += val & mask;
    }
    return sum;
};

cout << "С ветвлением: " << sum_with_branch(data) << endl;
cout << "Без ветвления: " << sum_without_branch(data) << endl;

return 0;
}

```

## Программа 6: Продвинутые техники

## 6.1. If constexpr и метапрограммирование

```
// =====
// Файл: 11_if_constexpr_advanced.cpp
// Описание: If constexpr и compile-time условия
// =====

#include <iostream>
#include <type_traits>
#include <string>
#include <vector>
using namespace std;

// Пример 1: Обработка разных типов
template<typename T>
auto process_value(T value) {
    // C++17: if constexpr - ветвление на этапе компиляции
    if constexpr (is_integral_v<T>) {
        cout << "Целое число: " << value << endl;
        return value * 2;
    }
    else if constexpr (is_floating_point_v<T>) {
        cout << "Число с плавающей точкой: " << value << endl;
        return value / 2.0;
    }
    else if constexpr (is_pointer_v<T>) {
        cout << "Указатель" << endl;
        if (value != nullptr) {
            return *value;
        }
        return T{}; // nullptr для указателей
    }
    else if constexpr (is_same_v<T, string>) {
        cout << "Строка: " << value << endl;
        return value + " (обработано)";
    }
    else {
        // static_assert обеспечит ошибку компиляции
        // для неподдерживаемых типов
        static_assert(false, "Неподдерживаемый тип");
    }
}
```

```
// Пример 2: Сериализация в зависимости от типа
template<typename T>
string serialize(const T& value) {
    if constexpr (is_arithmetic_v<T>) {
        return to_string(value);
    }
    else if constexpr (is_same_v<T, string>) {
        return "\"" + value + "\"";
    }
    else if constexpr (is_same_v<T, bool>) {
        return value ? "true" : "false";
    }
    else {
        // Для пользовательских типов
        return "custom_type";
    }
}
```

```
// Пример 3: Векторизованные операции (SIMD-подобные)
template<typename T>
class Vector {
private:
    T x, y, z;

public:
    Vector(T x, T y, T z) : x(x), y(y), z(z) {}

    template<typename U>
    auto dot(const Vector<U>& other) const {
        // Тип результата зависит от типов T и U
        if constexpr (is_floating_point_v<T> || is_floating_point_v<U>) {
            // Если хотя бы один тип с плавающей точкой
            return static_cast<double>(x * other.x) +
                static_cast<double>(y * other.y) +
                static_cast<double>(z * other.z);
        } else {
            // Оба целочисленных
            return x * other.x + y * other.y + z * other.z;
        }
    }
};
```

```
// Пример 4: Conditional compilation
template<int Version>
class Processor {
public:
    void process() {
        if constexpr (Version == 1) {
            cout << "Обработка версии 1" << endl;
            // Код только для версии 1
        }
        else if constexpr (Version == 2) {
            cout << "Обработка версии 2" << endl;
            // Код только для версии 2
        }
        else {
            static_assert(Version == 1 || Version == 2,
                          "Неподдерживаемая версия");
        }
    }
};

int main() {
    cout << "=== IF CONSTEXPR И METAPROGRAMMING ===\n" << endl;

    // Пример 1: Обработка разных типов
    cout << "1. Обработка разных типов:" << endl;

    auto r1 = process_value(42);
    cout << "Результат: " << r1 << endl << endl;

    auto r2 = process_value(3.14);
    cout << "Результат: " << r2 << endl << endl;

    int x = 100;
    auto r3 = process_value(&x);
    cout << "Результат: " << r3 << endl << endl;

    auto r4 = process_value(string("Hello"));
    cout << "Результат: " << r4 << endl << endl;

    // Пример 2: Сериализация
    cout << "\n2. Сериализация:" << endl;
    cout << "Число: " << serialize(42) << endl;
    cout << "Строка: " << serialize(string("test")) << endl;
```



```

    cout << "Bool: " << serialize(true) << endl;

    // Пример 3: Векторные операции
    cout << "\n3. Векторные операции:" << endl;
    Vector<int> v1(1, 2, 3);
    Vector<int> v2(4, 5, 6);
    Vector<double> v3(1.5, 2.5, 3.5);

    cout << "int . int = " << v1.dot(v2) << endl;
    cout << "int . double = " << v1.dot(v3) << endl;

    // Пример 4: Conditional compilation
    cout << "\n4. Conditional compilation:" << endl;
    Processor<1> p1;
    Processor<2> p2;

    p1.process();
    p2.process();

    // Processor<3> p3;  // Ошибка компиляции!

    return 0;
}

```

## Сборка и выполнение всех примеров

### Makefile для сборки всех примеров:

```

# =====
# Makefile для сборки всех примеров
# =====

CXX = g++
CXXFLAGS = -Wall -Wextra -std=c++17 -O2
TARGETS = all clean

# Все исполняемые файлы
EXECUTABLES = \
    01_basic_if_else \
    02_nested_if_dangling_else \

```

```

03_logical_operations \
04_complex_logical_expressions \
05_ternary_operator \
06_ternary_constexpr \
07_switch_basic \
08_switch_optimization \
09_patterns_antipatterns \
10_branch_prediction \
11_if_constexpr_advanced

```

```
# Правило по умолчанию
```

```
all: $(EXECUTABLES)
```

```
# Автоматическое правило для сборки
```

```
%.cpp
```

```
$(CXX) $(CXXFLAGS) $< -o $@
```

```
# Запуск всех примеров по очереди
```

```
run: all
```

```
@echo "Запуск всех примеров..."
```

```
@for exe in $(EXECUTABLES); do \
```

```
echo " "; \
```

```
echo "=== Запуск $$exe ==="; \
```

```
./$$exe || true; \
```

```
done
```

```
# Очистка
```

```
clean:
```

```
rm -f $(EXECUTABLES) *.o
```

```
.PHONY: all clean run
```

## Инструкция по сборке и запуску:

1. **Сохраните** каждый пример в отдельный файл с соответствующим именем
2. **Сохраните** Makefile в ту же папку
3. **Соберите** все примеры:

```
make all
```

#### 4. **Запустите** все примеры:

```
make run
```

#### 5. **Или запустите** конкретный пример:

```
./01_basic_if_else
```

### **Требования:**

- Компилятор с поддержкой C++17 (GCC 7+, Clang 5+, MSVC 2017+)
- Стандартная библиотека C++

Это полный сборник всех примеров из теоретического объяснения. Каждая программа самодостаточна и может быть скомпилирована и запущена отдельно для изучения конкретных аспектов условных операторов в C++.